

Requirements Engineering, Scope, and Project Planning

COMP-7431: Advanced Software Engineering | Week 3
Software Architect · Design Thinking · AI Periodic Table

Turn discovery evidence into a small, testable MVP promise.

Monday Morning: Put the Week 2 Evidence on the Table

WHAT THE TEAM BRINGS

1. Stakeholder and current-work evidence
2. Hopes, fears, and assumptions
3. Affinity clusters and empathy map
4. Needs, pain points, and open questions

WHAT THE ARCHITECT ADDS

5. Problem statement and target outcome
6. Functional and quality requirements
7. Acceptance criteria and priorities
8. Milestones, MVP boundary, and risks

Week 3 begins with a handoff: every important commitment must point back to evidence.

10:00 AM: The Sponsor Asks, “Can We Start the MVP?”

THE PRESSURE

“We understand the problem now. Let’s build the chatbot this week.”

THE ARCHITECT’S RESPONSE

- Yes—but first define the smallest valuable journey
- Name what success and failure look like
- Protect privacy, accessibility, provenance, and escalation
- Put the work, owners, and evidence into a reviewable backlog

Starting the MVP does not mean starting every feature. It means starting one defensible promise.

Week 3 Produces a Delivery Baseline the Team Can Verify

By the end of this lecture, you should be able to:

- Translate evidence into functional and quality requirements
- Write user stories, use cases, and measurable acceptance criteria
- Define AI-specific data, grounding, guardrail, and escalation needs
- Prioritize by value, risk, learning, dependency, and effort
- Draw an in-scope, out-of-scope, and deferred MVP boundary
- Plan milestones and preserve requirements in version control

The output is Requirements Specification v1 plus a prioritized, reviewable delivery plan.

Three Lenses Turn Discovery into the Same Delivery Decision

SOFTWARE ARCHITECT

- Make commitments explicit
- Expose quality and risk
- Protect system boundaries
- Delay irreversible choices

DESIGN THINKING

- Preserve the human need
- Find the opportunity
- State a testable hypothesis
- Invite feedback early

AI PERIODIC TABLE

- Identify required data
- Select justified AI elements
- Specify interaction and integration
- Bound autonomy and automation

All three lenses ask: what is the smallest responsible experiment that can produce useful evidence?

Design Thinking Finds the Opportunity

Week 2 asked what is happening. Week 3 selects one opportunity worth testing.

- Keep the need separate from the proposed interface
- Turn pain points into outcomes—not feature votes
- State assumptions the MVP must test
- Use acceptance criteria to design the experiment
- Let feedback revise the requirement

CampusConnect Enters Week 3 with Evidence—not Answers

DISCOVERY FINDING

Students cannot reliably identify the correct campus service when names and guidance overlap.

EVIDENCE WE MUST HONOR

- Misrouting creates delay and repeated explanations
- Conflicting content reduces trust
- High-risk topics require human escalation
- Accessibility and privacy affect the core journey
- Content ownership remains partly unresolved

A requirement may narrow the promise, but it may not erase inconvenient evidence.

The Problem Names the Barrier; the Outcome Names Progress

PROBLEM STATEMENT

Students seeking campus services cannot reliably identify the responsible office or next step when service language overlaps, causing delay, repeated contacts, and lost trust.

DESIRED OUTCOME

A student can describe a need in ordinary language, receive a traceable next step, correct a poor route, and reach a person when automation should stop.

The problem statement protects purpose; the outcome begins to define what the MVP must prove.

Requirements Form a Traceable Hierarchy—not a Flat List

A useful hierarchy connects:

- Outcome — the progress the product must enable
- Capability — a coherent ability the system provides
- Requirement — behavior, quality, rule, data, or constraint
- User story or use case — a reviewable slice of the interaction
- Acceptance criterion — observable evidence of success
- Test or demonstration — the recorded proof

Mini example: less misrouting → traceable routing → source citation → criterion → test case.

Different Requirement Types Answer Different Questions

Requirement type	CampusConnect example	Acceptance evidence
Functional	Recommend a responsible service and next step	End-to-end scenario test
Quality attribute	Show a usable response within the agreed latency target	Measured performance run
Business rule	High-risk or account-specific requests go to approved human help	Policy scenario review
Data / AI	Use approved sources; cite them or abstain	Grounding and source tests
Constraint	Do not store unnecessary student identifiers	Data-flow and log review

A complete MVP needs behavior, quality, rules, data, and constraints—not merely features.

Behavior, Quality, and Constraints Must Travel Together

FUNCTIONAL BEHAVIOR

- Accept an everyday-language request
- Recommend one responsible service
- Show a source and next step
- Allow correction or escalation

QUALITY ATTRIBUTES

- Accessible interaction
- Predictable response time
- Traceable content
- Recoverable failure
- Maintainable source ownership

BOUNDARIES

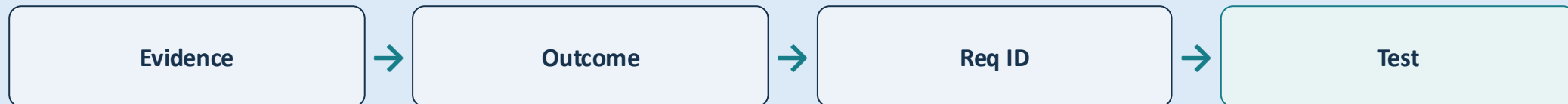
- Approved services only
- Minimum necessary data
- No autonomous account action
- Human review for high-risk topics
- One institution and language in MVP

If one story ignores quality or boundaries, the team will discover them later as defects.

Traceability Connects Evidence to Delivery

Keep one simple chain for every important commitment:

Each link keeps the reason, owner, and acceptance evidence visible.



A User Story Starts the Conversation

Story part	CampusConnect example
Role	Student who is unsure which office owns the issue
Need	Receive one responsible service and a clear next step
Value	Act without repeating the entire story
Conversation	Sources, uncertainty, correction, access, and escalation
Confirmation	Acceptance criteria and representative tests

The story card stays short. The team adds examples, rules, quality needs, and acceptance evidence through conversation.

Good Stories Carry Need, Rules, and Confirmation

STORY

As a student unsure who owns my issue, I want a reliable next step so I can act without repeating my story.

CONVERSATION

- Approved sources
- Overlapping services
- Source date
- Keyboard flow
- Correction
- High-risk escalation

CONFIRMATION

Representative requests reach an approved service, show the source and next step, preserve the original request, and allow correction or escalation.

The story is small; the shared understanding around it is rich.

Use Cases Expose the Complete Interaction

A use case adds the system boundary and alternate paths:

- Primary actor and desired outcome
- Trigger and preconditions
- Main success path
- Alternate and failure paths
- Business rules and quality conditions
- Postconditions, logs, and audit evidence

Mini example: a plausible but unsupported route is an alternate path—not an acceptable answer.

Acceptance Criteria Define Success Before Code

Acceptance criteria answer: what must be true for the team and stakeholder to accept this story?

- Use concrete conditions and observable outcomes
- Include boundaries, error behavior, and accessibility
- Name data, source, or policy expectations when relevant
- Avoid implementation details unless they are true constraints
- Keep each criterion focused enough to test or demonstrate

A criterion is not a coding instruction; it is a shared description of acceptable behavior.

Criteria Cover Success, Boundaries, and Failure

Scenario	Given / When / Then	Why it matters
Clear request	Given an approved service, when submitted, then show service, source, next step.	Happy path
Two plausible services	When evidence is close, then show clarification—not a guess.	Ambiguity
No approved source	When retrieval is empty, then abstain and offer human help.	Grounding
Student rejects route	When corrected, then preserve the request and allow another path.	Recovery
Keyboard journey	Given keyboard-only use, then every core action remains operable.	Accessibility

One story needs several examples because AI and workflow behavior changes at the edges.

Comfortable Words Hide Requirements Defects

Review each important sentence for:

- Vague adjectives: fast, easy, accurate, seamless, secure
- Undefined actors: user, admin, staff, system
- Hidden conditions: usually, when needed, if appropriate
- Combined behavior: “and,” “or,” multiple outcomes
- Missing units, thresholds, source sets, or time windows
- Unstated exceptions, recovery, ownership, or audit evidence

Rewrite “notify promptly” as who is notified, by which channel, after what event, and within what time.

Quality Requirements Need a Measure, Method, and Owner

Weak quality phrase	Measurable rewrite
Responses should be fast	95% of the validation requests display a result within 2.5 seconds
Routing should be accurate	At least 45 of 50 approved validation cases reach the accepted service
The app must be secure	No student identifier enters model context; logs redact submitted identifiers
The interface is accessible	The core journey meets agreed WCAG 2.2 AA criteria and keyboard tests
Answers should be trustworthy	Every recommendation displays an approved source or abstains

Targets below are examples for negotiation—not claims that CampusConnect has already achieved them.

An AI Feature Needs a Requirement Packet—not One Story

FUNCTIONAL CONTRACT

Given a student description, return a service ID, plain-language reason, approved source, next step, confidence state, and escalation option.

QUALITY AND SAFETY CONTRACT

Use approved content only; do not invent policy; protect identifiers; remain keyboard operable; clarify, abstain, or escalate when evidence is insufficient.

ACCEPTANCE EVIDENCE

Representative route cases

- grounding checks
- ambiguity cases
- failure recovery
- accessibility review
- latency run
- stakeholder playback

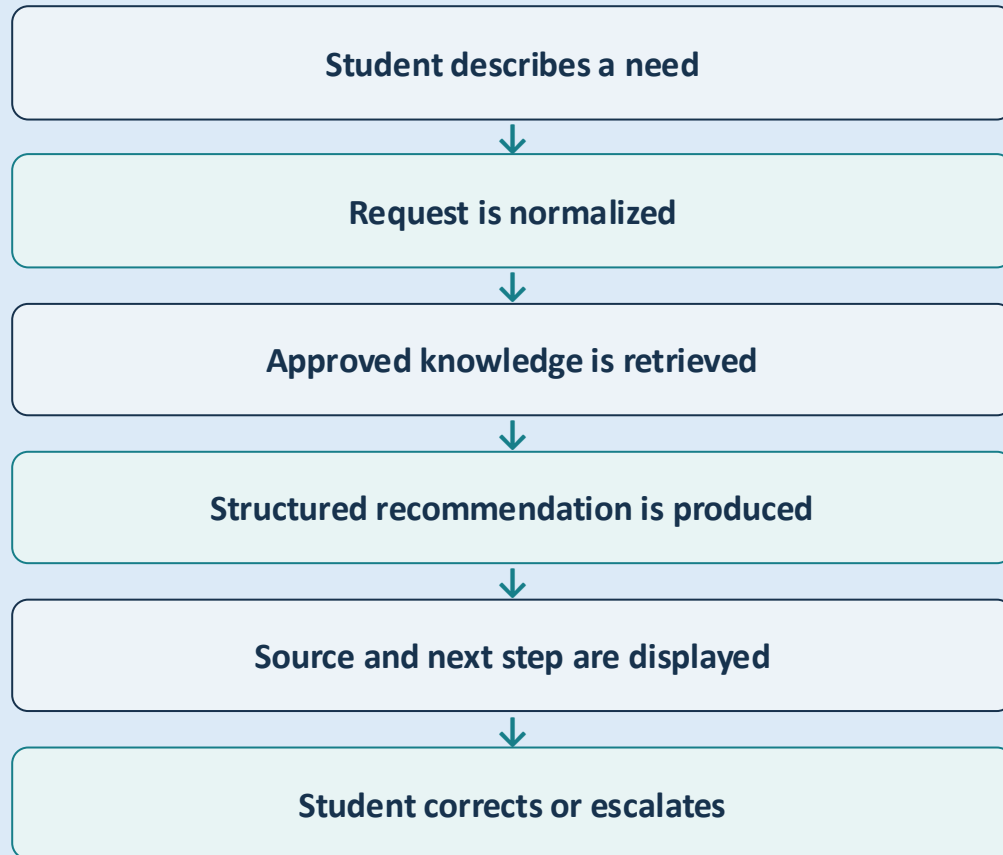
Failure Paths and Handoffs Are Requirements

The MVP must say what happens when:

- No approved source is retrieved
- Two services appear equally plausible
- A source is expired, missing, or contradictory
- The model or external API is unavailable
- A request contains sensitive or high-risk content
- The student rejects the recommendation
- Accessibility or language needs change the journey

A system that works only on the happy path is not yet an end-to-end MVP.

One Vertical Slice Carries the Requirement End to End



EVIDENCE TO RECORD

- Requirement IDs and issue link
- Representative inputs
- Retrieved source IDs
- Structured output
- Correction or escalation result
- Timing, errors, and review decision

A Requirement Review Looks for More Than Good Grammar

NECESSARY AND GROUNDED

- Traces to evidence, policy, or outcome
- Names a real stakeholder or system need
- Belongs in the current product boundary
- Avoids solution detail unless constrained

CONSISTENT AND FEASIBLE

- Does not conflict with another requirement
- Fits the known platform and schedule
- Exposes dependencies and unresolved decisions
- Can be decomposed into reviewable work

VERIFIABLE AND OWNED

- Uses observable language
- Includes success, failure, and edge conditions
- Has an acceptance method and owner
- Can change through a visible process

A requirement is review-ready when the team can explain why it exists and how acceptance will be decided.

Controlled Change Preserves Trust as Evidence Improves

A practical change process records:

- What evidence, policy, or constraint changed
- Which requirement, story, criterion, test, and milestone are affected
- Whether scope, cost, risk, or architecture must change
- Who owns the decision and who must review it
- The new version, rationale, and unresolved consequences

Modern does not mean undocumented; it means changes remain reviewable and traceable.

Requirements Specification v1 Is Short Enough to Use

1. Problem statement and target outcomes
2. Stakeholders, current journey, and evidence references
3. In-scope, out-of-scope, and deferred boundary
4. Functional, quality, business-rule, data, and AI requirements
5. User stories, use cases, and acceptance criteria
6. Assumptions, dependencies, risks, and open questions
7. Traceability to tests, milestones, and owners
8. Version and change history

If the specification cannot guide backlog review, testing, and architecture, it is documentation without leverage.

The AI Periodic Table Becomes a Requirements Question Map

KNOWLEDGE FOUNDATION

Data sources, preparation, and representations

Which documents, owners, dates, labels, chunks, embeddings, retrieval rules, and provenance are required?

INTELLIGENCE + INTERACTION

Models, prompts, context, guardrails, APIs, and tools

What output contract, quality target, failure behavior, privacy boundary, and integration rule must hold?

AGENTIC + APPLIED

Goals, planning, tool use, frameworks, automation, and applied systems

What may act, what needs approval, what must be logged, and where does human authority remain?

A Periodic Table cell enters the MVP only when its value, data, risk, and acceptance evidence are named.

Each AI Element Creates Work and Evidence

Periodic Table element	Requirement it creates	Acceptance evidence
Documents / enterprise data	Approved source registry, owner, date, retention	Provenance review
Parsing / chunking / retrieval	Usable sections and repeatable retrieval behavior	Retrieval case set
Prompt / context / guardrails	Structured response, abstention, safe boundaries	Output evaluation
API / MCP / webhooks	Schema, authentication, errors, retry, timeout	Integration tests
Agent / planning / tool use	Permission, approval, stopping, logging, handoff	Scenario and audit review

Scope Protects the Core Outcome from Unlimited Possibilities

IN THE FIRST MVP

- Five approved campus services
- Everyday-language request
- Retrieval from owned sources
- One traceable recommendation
- Correction and human escalation
- Accessible core journey
- Basic evaluation and logging

OUT OF SCOPE OR DEFERRED

- Autonomous account transactions
- Every department and policy
- Voice and multilingual support
- Personalized records or advising
- Multi-agent orchestration
- Institution-wide production rollout
- High-scale operations

Out of scope does not mean unimportant. It means the first release will not pretend to deliver it.

AI Can Critique Requirements—but It Cannot Accept Them

Engineering activity	Useful AI assistance	Required human control
Requirement drafting	Suggest clearer wording and missing fields	Verify evidence, meaning, and owner
Acceptance criteria	Generate boundary and failure examples	Confirm expected behavior
Ambiguity review	Flag vague or combined statements	Resolve with authoritative context
Quality review	Suggest missing quality attributes	Negotiate targets and methods
Backlog splitting	Propose vertical slices and dependencies	Protect the stakeholder outcome
Risk analysis	Generate failure scenarios	Assess consequence and response
Plan communication	Format milestones and summaries	Own estimates and commitments

Prompt and Context Engineering Become Specification Work

CONTEXT PACKET

Problem statement • evidence IDs • approved terminology • current scope • quality targets • constraints
• examples • unresolved questions

REQUIREMENTS-CRITIC PROMPT

Use only the supplied context. For each requirement, flag ambiguity, missing failure behavior, missing NFRs, and unsupported assumptions. Cite evidence IDs. Do not add features.

HUMAN DECISION RECORD

- Accepted
- revised
- rejected
- needs stakeholder evidence
- owner
- issue or specification change
- Engineering/AI Use Log entry

POC, Prototype, Spike, and MVP Differ

Artifact	Question it answers
Problem statement	Which barrier and outcome matter?
Proof of concept	Can approved-source retrieval work at all?
Prototype	Will students understand and trust the journey?
Technical spike	Can one risky API or latency target be met?
MVP	Can a small end-to-end product deliver and measure value?

Do not call every early artifact an MVP. Name the question it is built to answer.

CampusConnect's MVP Is One End-to-End Learning Loop

The first slice proves one useful student journey and exposes the biggest uncertainties:

- Can approved content support trustworthy routing?
- Can the interface explain and recover from uncertainty?
- Can the team maintain the sources and review evidence?



Prioritization Balances Value, Risk, Learning, and Dependency

A responsible priority decision considers:

- Stakeholder value and consequence of delay
- Evidence strength behind the need
- Safety, privacy, accessibility, and uncertainty
- Learning value: which work reduces a major unknown?
- Dependencies: what must exist before something else can work?
- Effort, capacity, and opportunity cost
- A written rationale—not only a High label

A priority score is a conversation aid. It does not replace judgment or stakeholder responsibility.

Milestones Are Evidence Gates—not Decorative Dates

1. Discovery accepted — problem, outcome, evidence, and unknowns reviewed
2. Baseline accepted — requirements, NFRs, criteria, and scope reviewed
3. Technical risk reduced — highest-risk spike produces evidence
4. Vertical slice demonstrated — core journey works end to end
5. MVP accepted — criteria, quality evidence, and stakeholder playback complete
6. Next decision recorded — expand, revise, or stop

A milestone states what evidence will exist and who will review it—not merely a date on a timeline.

Estimates Express Uncertainty; Scope Absorbs New Evidence

FALSE CERTAINTY

“The integration will take exactly 14 hours.”



EVIDENCE-BASED FORECAST

“Most likely 2–4 days after the API and test credentials are available; longer if the response schema or access policy changes.”

WHAT MAKES THE SECOND STRONGER

- Names assumptions and dependencies
- Uses a range instead of false precision
- Identifies conditions that could change the forecast
- Creates a trigger for re-estimation and scope trade-offs

CampusConnect’s Backlog Has a Reasoned Order

Backlog item	Why it comes here
Confirm five services and source owners	Dependency for every grounded result
Build one-service routing slice	Highest-value end-to-end learning
Add source citation and correction	Trust and recoverability are core
Add high-risk human escalation	High consequence of unsafe behavior
Verify keyboard and screen-reader journey	Quality belongs in the first slice
More departments, voice, multi-agent	Deferred until the core pattern earns expansion

The first items reduce foundational risk and create one complete slice before the team expands breadth.

The Requirements Toolchain Creates One Engineering Trail

Artifact or tool	Specific role in Week 3
Repository Markdown	Version problem statement and Requirements v1
GitHub Issues / forms	Capture stories, criteria, NFRs, owner, and evidence IDs
GitHub Projects	Custom fields, backlog, board, iterations, and roadmap
Jira	Work items, dependencies, acceptance fields, and milestones
Trello	Lightweight cards, checklists, labels, dates, and automation
Branch + pull request	Review requirement changes before merging
Engineering/AI Use Log	Record prompt, output, critique, and human decision

Tools differ, but the artifacts should link from evidence to issue, review, implementation, and acceptance.

The Lab Produces a Buildable Delivery Baseline

REQUIREMENTS PACKAGE

1. Evidence-backed problem statement
2. Functional and quality requirements
3. User stories and acceptance criteria
4. AI, data, privacy, accessibility, and failure requirements
5. Requirements Specification v1

SCOPE AND PLAN

6. In / out / deferred MVP boundary
7. Prioritized backlog with rationale
8. Owners, dependencies, estimates, and milestones
9. GitHub Projects, Jira, or Trello board
10. Updated Engineering/AI Use Log

The package should be small enough to review in one meeting and complete enough to guide the first architecture decision.

Choose One AI Critic and One Planning Surface—Then Review

“The cool tool is the one that makes reasoning visible, not the one that produces the most text.”

AI CRITIC OPTIONS

- GitHub Spec Kit for a spec-driven workflow
- GitHub Copilot with repository instructions
- Jira with Rovo for clarity, criteria, and edge cases
- ChatGPT or Claude with the product critic prompt

PLANNING OPTIONS

- GitHub Projects for repo-linked work
- Jira for richer work-item workflows
- Trello for a lightweight board and automation
- Miro AI for a user-story map before issue creation

Do not use every tool. Select the smallest toolchain your team can explain and maintain.

Requirements Belong in Version Control Beside the Work

REPOSITORY ARTIFACTS

- docs/problem-statement.md
- docs/requirements-v1.md
- specs/001-routing/
- .github/ISSUE_TEMPLATE/user-story.yml
- docs/engineering-ai-use-log.md

REVIEW TRAIL

- Branch: docs/req-routing-v1
- Commit: docs(req): define source-citation criteria
- Pull request links issues and evidence
- Peer checks ambiguity and scope
- Merge records the accepted baseline

A requirements change deserves the same visible reasoning and review as a code change.

Lab Preview: Build the Backlog That Starts the MVP

“Your backlog is not a parking lot of ideas. It is the executable argument for what the team will build first.”

FIRST, DEFINE

- Reopen Week 2 evidence
- Refine the problem and target outcome
- Write functional and non-functional requirements
- Add failure, accessibility, data, and AI criteria
- Draw the MVP boundary

THEN, PLAN

- Convert requirements into stories and issues
- Prioritize with rationale
- Add owners, dependencies, estimates, and milestones
- Run an AI requirements critique
- Record human corrections and AI use

The lab should end with Requirements Specification v1 and a board ready for the first vertical slice.

Look Ahead: Week 4 Compares Architecture Alternatives

WEEK 3 HANDOFF

1. Problem statement and outcomes
2. Functional, quality, data, and AI requirements
3. Acceptance criteria and failure behavior
4. MVP boundary and vertical slice
5. Priorities, milestones, dependencies, and risks

WEEK 4 QUESTIONS

6. Which responsibilities belong in which component?
7. Where should data live and who owns it?
8. Which integration style fits the journey?
9. Which architecture best satisfies the NFRs?
10. What trade-offs need a spike or decision record?

Week 3 defines the promise. Week 4 compares the designs capable of keeping it.